

SESSION 10 — Merging & Joining DataFrames

Why Merging is Important

In real-world scenarios, data is often stored in **multiple tables/files**.

Examples:

Customer Table:

Customer_ID	Name
1	Aman
2	Priya

Sales Table:

Customer_ID	Amount
1	500
2	700

To analyze data properly, we need to **combine these tables**.

Solution

Pandas provides:

`merge()`

Key Concept

Merge is similar to **SQL JOIN**

✓ `merge()` Function

Syntax

```
pd.merge(left_df, right_df, how="type", on="column")
```

Parameters

`left_df` → first DataFrame

`right_df` → second DataFrame

`how` → type of join:

- "inner" → common values only
- "left" → all values from left
- "right" → all values from right
- "outer" → all values from both

`on` → column name used for joining

Use Case

- Combine customer data with sales
- Combine datasets from multiple sources

```
import pandas as pd

customers = pd.DataFrame({
    "Customer_ID": [1,2,3],
    "Name": ["Aman","Priya","Rahul"]
})

sales = pd.DataFrame({
    "Customer_ID": [1,2,4],
    "Amount": [500,700,300]
})

customers, sales
```

```
(   Customer_ID  Name
0             1  Aman
1             2  Priya
2             3  Rahul,
   Customer_ID  Amount
0             1    500
1             2    700
2             4    300)
```

✓ 1. Inner Join

Keeps only matching values in both tables.

```
pd.merge(customers, sales, how="inner", on="Customer_ID")
```

	Customer_ID	Name	Amount
0	1	Aman	500
1	2	Priya	700

✓ 2. Left Join

Keeps all rows from left table.

```
pd.merge(customers, sales, how="left", on="Customer_ID")
```

	Customer_ID	Name	Amount
0	1	Aman	500.0
1	2	Priya	700.0
2	3	Rahul	NaN

3. Right Join

Keeps all rows from right table.

```
pd.merge(customers, sales, how="right", on="Customer_ID")
```

	Customer_ID	Name	Amount
0	1	Aman	500
1	2	Priya	700
2	4	NaN	300

4. Outer Join

Keeps all rows from both tables.

```
pd.merge(customers, sales, how="outer", on="Customer_ID")
```

	Customer_ID	Name	Amount
0	1	Aman	500.0
1	2	Priya	700.0
2	3	Rahul	NaN
3	4	NaN	300.0

Merge on Multiple Columns

Syntax

```
pd.merge(df1, df2, on=["col1","col2"])
```

Use Case

When one column is not enough to uniquely identify data.

```
df1 = pd.DataFrame({
    "ID": [1, 1, 2],
    "City": ["Delhi", "Mumbai", "Delhi"],
    "Sales": [100, 200, 300]
})

df2 = pd.DataFrame({
    "ID": [1, 2],
    "City": ["Delhi", "Delhi"],
    "Target": [150, 350]
})

pd.merge(df1, df2, on=["ID", "City"])
```

	ID	City	Sales	Target
0	1	Delhi	100	150
1	2	Delhi	300	350

Default Behavior of pd.merge()

When you do:

```
pd.merge(df1, df2, on=["ID", "City"])
```

and do NOT specify `how`

Default Join Type

The default join is:

INNER JOIN

What is INNER JOIN?

It keeps only those rows where:

- Values match in BOTH DataFrames
- Based on the given key(s)

Important Rule

If no match exists → row is dropped

Syntax Reminder

```
pd.merge(df1, df2, how="inner", on=["ID", "City"])
```

is SAME as:

```
pd.merge(df1, df2, on=["ID", "City"])
```

✓ Demo — Customer + Sales Merge

Goal:

Combine customer details with their sales data.

```
merged_df = pd.merge(customers, sales, how="left", on="Customer_ID")
```

merged_df

	Customer_ID	Name	Amount
0	1	Aman	500.0
1	2	Priya	700.0
2	3	Rahul	NaN

Question 1 — Default Merge Trap

Create two DataFrames:

df1:

ID	Name
1	Aman
2	Priya
3	Rahul

df2:

ID	Score
2	80
3	90
4	70

Tasks:

- 1 Perform merge WITHOUT specifying `how`
- 2 Explain which rows will appear and why
- 3 How many rows will be in output?

Question 2 — Duplicate Key Explosion

Create:

df1:

ID	Product
1	A
1	B

df2:

ID	Price
1	100
1	200

Tasks:

- 1 Perform inner merge
- 2 Count number of rows in output
- 3 Explain why rows increase

(Hint: Cartesian multiplication)

Question 3 — Left Join with Missing Values

Create:

df1:

ID	Name
1	Aman
2	Priya
3	Rahul

df2:

ID	Salary
1	50000

Tasks:

- 1 Perform left join
 - 2 Identify rows with missing Salary
 - 3 Fill missing Salary with 0
-

Question 4 — Multiple Key Merge Trap

Create:

df1:

ID	City	Sales
1	Delhi	100
1	Mumbai	200

df2:

ID	City	Target
1	Delhi	150

Tasks:

- 1 Merge using only ID
- 2 Merge using ID + City
- 3 Compare both outputs

Explain:

Why results are different?

Question 5 — Real-World Merge Scenario

Create:

customers:

Customer_ID	Name
1	Aman
2	Priya
3	Rahul

orders:

Customer_ID	Order_Amount
1	500
1	700
2	300

Tasks:

- 1 Merge both datasets
- 2 Calculate total order amount per customer
- 3 Identify customers with no orders
- 4 Replace missing values with 0

SESSION 11 — Concatenation & Data Reshaping

Why Concatenation is Needed

Sometimes data is split across multiple files:

Example:

January Sales

February Sales

March Sales

We need to **combine them into one dataset**.

Solution

`pd.concat()`

✓ `pd.concat()`

Syntax

`pd.concat([df1, df2], axis=0)`

Parameters

axis:

0 → row-wise (default)

1 → column-wise

ignore_index:

True → reset index

Use Cases

- Combine datasets
- Append rows
- Merge columns

```
df1 = pd.DataFrame({"Sales":[100,200]})  
df2 = pd.DataFrame({"Sales":[300,400]})  
  
pd.concat([df1, df2])
```

	Sales
0	100
1	200
0	300
1	400

```
pd.concat([df1, df2], ignore_index=True)
```

	Sales
0	100
1	200
2	300
3	400

✓ Column-wise Concatenation

axis = 1

```
df1 = pd.DataFrame({"A":[1,2]})  
df2 = pd.DataFrame({"B":[3,4]})  
  
pd.concat([df1, df2], axis=1)
```

	A	B
0	1	3
1	2	4

✓ Pivot

Purpose

Reshape data from long format to wide format.

Syntax

`df.pivot(index, columns, values)`

```
data = {
    "Month": ["Feb", "Feb", "Jan", "Jan"],
    "Product": ["A", "B", "A", "B"],
    "Sales": [100, 200, 150, 250]
}

df = pd.DataFrame(data)

df.pivot(index="Month", columns="Product", values="Sales")
```

Product	A	B
Month		
Feb	100	200
Jan	150	250

✓ Pivot Table

Purpose

Similar to pivot but supports aggregation.

Syntax

`df.pivot_table(index, columns, values, aggfunc)`

```
df.pivot_table(index="Month", columns="Product", values="Sales", aggfunc="sum")
```

Product	A	B
Month		
Feb	150	250
Jan	100	200

✓ Pivot

What Problem Does Pivot Solve?

Sometimes data is stored in **long format**:

Month	Product	Sales
Jan	A	100
Jan	B	200

Month	Product	Sales
Feb	A	150
Feb	B	250

This is called **long / tidy format**.

What Do We Want?

We want something like:

Month	A	B
Jan	100	200
Feb	150	250

This is called **wide format**.

What Pivot Does

Pivot rearranges data:

Rows → become index

Unique column values → become new columns

Values → fill the table

Core Idea

pivot = "reshape table"

```
import pandas as pd

# Step 1: Create long format dataset
data = {
    "Month": ["Jan", "Jan", "Feb", "Feb", "Mar", "Mar"],
    "Product": ["A", "B", "A", "B", "A", "B"],
    "Sales": [100, 200, 150, 250, 300, 400]
}

df = pd.DataFrame(data)

print("Original Data (Long Format):\n")
print(df)
```

Original Data (Long Format):

	Month	Product	Sales
0	Jan	A	100
1	Jan	B	200
2	Feb	A	150
3	Feb	B	250
4	Mar	A	300
5	Mar	B	400

✓ Step-by-Step Pivot Explanation

We want:

- Month → rows
- Product → columns
- Sales → values

Syntax

```
df.pivot(index="Month", columns="Product", values="Sales")
```

Parameter Meaning

index → what becomes rows

columns → what becomes columns

values → what fills the table

```
pivot_df = df.pivot(index="Month", columns="Product", values="Sales")

print("\nPivot Output (Wide Format):\n")
print(pivot_df)
```

Pivot Output (Wide Format):

Product	A	B
Month		
Feb	150	250
Jan	100	200
Mar	300	400

✓ Output Understanding

Month	A	B
Jan	100	200
Feb	150	250
Mar	300	400

How It Worked

For each combination:

Month + Product → value is picked from Sales

Example:

Month = Jan, Product = A → 100

Month = Jan, Product = B → 200

Important Rule (VERY IMPORTANT)

Pivot works ONLY when:

Each (index, column) pair has ONE value

If duplicates exist → ERROR

```
# Duplicate example (WILL FAIL)

data = {
    "Month": ["Jan", "Jan"],
    "Product": ["A", "A"],
    "Sales": [100, 150]
}

df = pd.DataFrame(data)

# This will raise error
df.pivot(index="Month", columns="Product", values="Sales")
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_251/216704865.py in <cell line: 0>()
     10
     11 # This will raise error
--> 12 df.pivot(index="Month", columns="Product", values="Sales")

----- 5 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/reshape/reshape.py in _make_selectors(self)
    208
    209         if mask.sum() < len(self.index):
--> 210             raise ValueError("Index contains duplicate entries, cannot reshape")
    211
    212         self.group_index = comp_index

ValueError: Index contains duplicate entries, cannot reshape
```

Why Pivot Fails Here?

Because:

Month = Jan, Product = A has TWO values:

100 and 150

- Pivot does not know which one to choose

Solution

Use `pivot_table()` instead

```
df.pivot_table(index="Month", columns="Product", values="Sales", aggfunc="sum")
```

Product	A
Month	
Jan	250

Pivot vs Pivot Table

Feature	pivot	pivot_table
Handles duplicates	No	Yes

Feature	pivot	pivot_table
Aggregation	No	Yes
Default aggfunc	No	mean

Default Parameters (Important)

`pivot()`:

```
df.pivot( index=None, columns=None, values=None )
```

No defaults are useful → you MUST specify all 3

`pivot_table()`:

```
df.pivot_table( index=None, columns=None, values=None, aggfunc="mean" )
```

Default aggregation = mean

Real-World Use Case

Imagine:

You have sales data like:

Date | Product | Sales

And you want:

Date-wise report:

Date | A | B | C

Use pivot()

Where It Is Used

- Business reports
- Dashboard creation
- Excel-like summaries
- Data reshaping before ML

✓ Demo — Combine Monthly Sales

Goal:

Combine multiple monthly sales datasets.

```
jan = pd.DataFrame({"Sales": [100, 200]})
feb = pd.DataFrame({"Sales": [150, 250]})

combined = pd.concat([jan, feb], ignore_index=True)

combined
```

Sales	
0	100
1	200
2	150
3	250

Question 1 — Basic Merge Practice

Create two DataFrames:

customers:

Customer_ID	Name
1	Aman
2	Priya
3	Rahul

orders:

Customer_ID	Amount
1	500
2	700
4	300

Tasks:

- 1 Perform inner merge
- 2 Perform left merge
- 3 Perform outer merge
- 4 Compare number of rows in each result

Question 2 — Merge and Analyze

Using the same dataset:

- 1 Perform left join
- 2 Identify customers with missing order values
- 3 Replace missing values with 0
- 4 Calculate total order amount

Question 3 — Multiple Key Merge

Create:

df1:

ID	City	Sales
1	Delhi	100
1	Mumbai	200
2	Delhi	300

df2:

ID	City	Target
----	------	--------

1	Delhi	150
---	-------	-----

2	Delhi	350
---	-------	-----

Tasks:

1 Merge using ID only

2 Merge using ID and City

3 Compare both outputs

Question 4 — Duplicate Key Scenario

Create:

df1:

ID	Product
----	---------

1	A
---	---

1	B
---	---

df2:

ID	Price
----	-------

1	100
---	-----

1	200
---	-----

Tasks:

1 Perform inner merge

2 Count number of rows

3 Explain why rows increased

Question 5 — Concat Row-wise

Create two DataFrames:

df1:

Sales = [100, 200]

df2:

Sales = [300, 400]

Tasks:

1 Concatenate row-wise

2 Concatenate using ignore_index=True

3 Compare both outputs

Question 6 — Concat Column-wise

Create:

df1:

A = [1, 2]

df2:

B = [3, 4]

Tasks:

- 1 Concatenate column-wise
 - 2 Observe index alignment
 - 3 Explain output
-

Question 7 — Combine Monthly Data

Create three DataFrames:

Jan = [100, 200]

Feb = [150, 250]

Mar = [300, 350]

Tasks:

- 1 Combine all months into one DataFrame
 - 2 Reset index
 - 3 Add a new column Month indicating source
-

Question 8 — Basic Pivot

Create dataset:

Month	Product	Sales
Jan	A	100
Jan	B	200
Feb	A	150
Feb	B	250

Tasks:

- 1 Apply pivot
 - 2 Set Month as index
 - 3 Set Product as columns
 - 4 Use Sales as values
-

Question 9 — Pivot Table

Using same dataset:

- 1 Create pivot_table
 - 2 Use sum as aggregation
 - 3 Compare pivot vs pivot_table
-

Question 10 — Full Workflow (Easy but Long)

Create:

customers:

ID	Name
1	Aman
2	Priya
3	Rahul

sales:

ID	Product	Amount
1	A	100
1	B	200
2	A	300

Tasks:

- 1 Merge customers and sales
- 2 Replace missing Amount with 0
- 3 Create pivot table:
 - index = Name
 - columns = Product
 - values = Amount
- 4 Fill missing pivot values with 0

SESSION 12 — GroupBy & Aggregations in Pandas

Why GroupBy is Important

In real-world data analysis, we often need to answer questions like:

- Total revenue per region
- Average salary per department
- Maximum sales per product

This requires:

- Grouping data
- Applying calculations

What is GroupBy?

GroupBy means:

- Split data into groups
- Apply a function
- Combine results

Real-Life Analogy

Think of:

Class of students → grouped by section

Then:

- Calculate average marks per section
- Find top student per section

One-Line Definition

GroupBy = Split + Apply + Combine

✓ Basic Syntax

```
df.groupby("column")
```

Important

groupby() does NOT compute anything immediately

It creates a grouped object.

You must apply functions like:

- sum()
- mean()
- count()
- max()
- min()

```
import pandas as pd

data = {
    "Region": ["North", "South", "North", "South", "East"],
    "Category": ["A", "A", "B", "B", "A"],
    "Revenue": [100, 200, 150, 250, 300]
}

df = pd.DataFrame(data)

df
```

	Region	Category	Revenue
0	North	A	100
1	South	A	200
2	North	B	150
3	South	B	250
4	East	A	300

✓ Example 1 — Group by One Column

Group data by Region

```
df.groupby("Region").sum()
```

	Category	Revenue
Region		
East	A	300
North	AB	250
South	AB	450

Parameters of groupby()

```
df.groupby(by, axis=0, level=None, as_index=True, sort=True, group_keys=True, observed=False, dropna=True)
```

Parameter Explanation

1. by

Column name(s) used for grouping

Example: "groupby('Region')"

2. axis (default = 0)

0 → group rows

1 → group columns

3. level

Used with MultiIndex (advanced)

4. as_index (default = True)

True → group column becomes index

False → group column remains normal column

5. sort (default = True)

True → sort group keys

False → keep original order

6. group_keys (default = True)

Controls whether group labels appear in output

7. observed (default = False)

Used with categorical data

8. dropna (default = True)

Drop missing group keys

```
# as_index=False example
```

```
df.groupby("Region", as_index=False).sum()
```

	Region	Category	Revenue
0	East	A	300
1	North	AB	250
2	South	AB	450

✓ Example 2 — Group by Multiple Columns

Group by Region and Category

```
df.groupby(["Region", "Category"]).sum()
```

		Revenue
Region	Category	
East	A	300
	B	150
North	A	100
	B	150
South	A	200
	B	250

✓ Aggregation Functions

Common functions:

- `sum()`
- `mean()`
- `count()`
- `max()`
- `min()`
- `std()`

Example

```
df.groupby("Region")["Revenue"].mean()
```

		Revenue
Region		
East		300.0
North		125.0
South		225.0

dtype: float64

✓ agg() Function

Allows multiple aggregations

Syntax

```
df.groupby("column").agg(function)
```

Example

```
df.groupby("Region").agg({  
    "Revenue": "sum"  
})
```

Revenue	
Region	
East	300
North	250
South	450

✓ Multiple Aggregations

We can apply multiple functions

```
df.groupby("Region").agg({  
    "Revenue": ["sum", "mean", "max"]  
})
```

Revenue			
	sum	mean	max
Region			
East	300	300.0	300
North	250	125.0	150
South	450	225.0	250

✓ Rename Aggregation Output

Using named aggregations

```
df.groupby("Region").agg(  
    Total_Revenue=("Revenue", "sum"),  
    Avg_Revenue=("Revenue", "mean")  
)
```

	Total_Revenue	Avg_Revenue
Region		
East	300	300.0
North	250	125.0
South	450	225.0

✓ Revenue by Region & Category

Goal:

Find total revenue grouped by Region and Category

```
df.groupby(["Region", "Category"])["Revenue"].sum()
```

		Revenue
Region	Category	
East	A	300
North	A	100
	B	150
South	A	200
	B	250

dtype: int64

✓ Reset Index (Important Step)

After grouping, index becomes hierarchical

Use:

reset_index()

```
# result = df.groupby(["Region", "Category"])["Revenue"].sum()
result = df.groupby(["Region", "Category"])["Revenue"].sum().reset_index()
```

result

	Region	Category	Revenue
0	East	A	300
1	North	A	100
2	North	B	150
3	South	A	200
4	South	B	250

Real-World Use Cases

- Sales analysis by region
 - Customer segmentation
 - Product performance
 - Financial reporting
-

Common Mistakes

- Forgetting aggregation after groupby
- Confusion with multi-index
- Not using reset_index()

Summary

GroupBy → grouping data

Aggregation → applying functions

agg() → multiple calculations

Multi-group → more granular analysis

Key Insight

GroupBy is the backbone of data analytics in Pandas

✓ Pandas Practice Set — GroupBy & Aggregations

Question 1

Create a DataFrame:

Region = ["North", "South", "North", "East"]

Revenue = [100, 200, 150, 300]

Tasks:

- Group by Region
 - Calculate total revenue
-

Question 2

Using the same dataset:

- Find average revenue per Region
-

Question 3

Create a dataset:

```
Category = ["A","A","B","B","A"]  
Revenue = [100,200,150,250,300]
```

Tasks:

- Group by Category
 - Find total revenue
-

Question 4

Using the same dataset:

- Find maximum revenue per Category
 - Find minimum revenue per Category
-

Question 5

Create a dataset:

```
Region = ["North","South","North","South","East"]  
Revenue = [100,200,150,250,300]
```

Tasks:

- Count number of rows per Region
-

Question 6

Create a dataset:

```
Region = ["North","South","North","East"]  
Category = ["A","A","B","B"]  
Revenue = [100,200,150,300]
```

Tasks:

- Group by Region and Category
 - Calculate total revenue
-

Question 7

Using the same dataset:

- Calculate mean revenue for each Region
-

Question 8

Create a dataset:

```
Region = ["North","South","North","East"]  
Revenue = [100,200,150,300]
```

Tasks:

- Use groupby with as_index=False
 - Compare output with default groupby
-

Question 9

Create a dataset:

Region = ["North","South","North","East"]

Revenue = [100,200,150,300]

Tasks:

- Sort groups in descending order of total revenue
-

Question 10

Create a dataset:

Region = ["North","South","North","South","East"]

Revenue = [100,200,150,250,300]

Tasks:

- Use agg() to calculate sum and mean
-

Question 11

Using the same dataset:

- Rename aggregated columns using named aggregation
-

Question 12

Create a dataset:

Region = ["North","South","North","South","East"]

Category = ["A","A","B","B","A"]

Revenue = [100,200,150,250,300]

Tasks:

- Group by Region and Category
 - Find total revenue
 - Reset index
-

Question 13

Create a dataset:

Region = ["North","South","North","East"]

Revenue = [100,200,150,300]

Tasks:

- Use groupby and find count of Revenue values
-

Question 14

Create a dataset:

```
Region = ["North","South","North","East"]
```

```
Revenue = [100,200,150,300]
```

Tasks:

- Find standard deviation of revenue per Region
-

Question 15

Create a dataset:

```
Region = ["North","South","North","South","East"]
```

```
Category = ["A","A","B","B","A"]
```

```
Revenue = [100,200,150,250,300]
```